Enable RDP hardware acceleration on a Linux VM in Microsoft Hyper-V – Michael Waterman

michaelwaterman.nl/2025/03/05/enable-rdp-hardware-acceleration-on-a-linux-vm-in-microsoft-hyper-v

Michael Waterman March 5, 2025

How to pass through a GPU and optimize remote performance in Ubuntu

Running a GPU-accelerated remote desktop on a Linux virtual machine (VM) in Microsoft Hyper-V can significantly improve performance for graphical applications, GPU intensive workloads, and even remote testing. However, Hyper-V does not support full PCI passthrough like VMware or Proxmox. Instead, it provides Discrete Device Assignment (DDA), which allows passing a GPU directly to a VM.

This guide walks you through passing through a NVIDIA GeForce GTX 1650 to a Ubuntu LTS VM and optimizing RDP with xrdp-egfx for hardware-accelerated performance. This is just the hardware setup that I have, it should work similarly on different configs, let's dive in.

Enabling GPU Passthrough (DDA) in Hyper-V

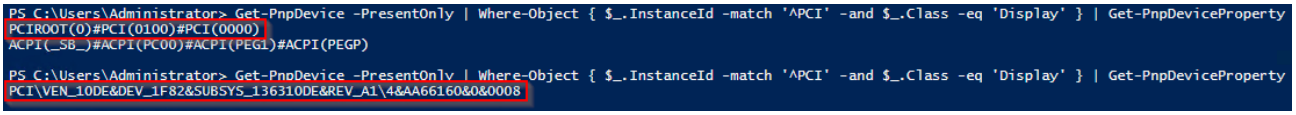
- Run PowerShell as Administrator on the Hyper-V host.
- List the InstanceId and the LocationPath of the GPU:

```
Get-PnpDevice -PresentOnly | Where-Object { $_.InstanceId -match '^PCI' -and
$_.Class -eq 'Display' } | Get-PnpDeviceProperty -KeyName
DEVPKEY_Device_LocationPaths | select -ExpandProperty Data
Get-PnpDevice -PresentOnly | Where-Object { $_.InstanceId -match '^PCI' -and
$_.Class -eq 'Display' } | Get-PnpDeviceProperty -KeyName DEVPKEY_Device_InstanceId
| select -ExpandProperty Data
```

The first command will get the "LocationPaths", the second one will get the "InstanceId", we'll need them both in the following steps. It will look something like this:

```
PCIROOT(0)#PCI(0100)#PCI(0000)
PCI\VEN_10DE&DEV_1F82&SUBSYS_136310DE&REV_A1\4&AA66160&0&0008
```

Note them both down, we'll need them in the following steps. (Yours may vary slightly, adjust accordingly.)



Disable the GPU on the host (to allow passthrough):

```
Disable-PnpDevice -InstanceId
'PCI\VEN_10DE&DEV_1F82&SUBSYS_136310DE&REV_A1\4&AA66160&0&0008' -Confirm:$false
```

When the command is successful, no feedback will be provided. In device manager you will notice that the device is disabled, but still visible, meaning available to the operating system, let's fix that.

Note! This will disable the GPU on your Hyper-V machine, if you have just one GPU installed, RDP will be the only option to connect remotely.

```
Dismount-VMHostAssignableDevice -LocationPath "PCIROOT(0)#PCI(0100)#PCI(0000)" -
Force
```

At this point the GPU will no longer be visible in device manager and is now available for passthrough to a VM.

GUI Based DDA

A couple of days ago I discovered a great GUI based tool on Reddit that can do all the stuff I explained in the previous chapter and more. If you're unfamiliar or uncomfortable using the command-line, check out this Github repro:

https://github.com/Justsenger/ExHyperV/blob/main/README_en.md

Create the Linux VM

Create a VM as you would normally do, install, configure and update the Linux machine and return to this guide for the next steps. As an example you can use the following PowerShell script to create a VM suitable for Ubuntu.

```
$Name = "Ubuntu"
$ISOPath = "E:\Operating Systems\Ubuntu\ubuntu-22.04.3-desktop-amd64.iso"
$SwitchName = "External Virtual Switch"
$NewVHDPath = (Join-Path -Path "D:\Virtual Hard Disks" -ChildPath
"Ubuntu_Disk_0.vhdx")

# Create the VM
New-VM -Name $Name `
       -MemoryStartupBytes 8GB `
       -Generation 2 `
       -NewVHDSizeBytes 127GB `
       -NewVHDPath $NewVHDPath `
       -SwitchName $SwitchName

# Set the number of virtual processors
Set-VM -Name $Name -ProcessorCount 8

# Enable Secure Boot for Ubuntu (if needed)
Set-VMFirmware -VMName $Name -EnableSecureBoot On -SecureBootTemplate
"MicrosoftUEFICertificateAuthority"

# Mount the ISO as a virtual DVD
```

```
Add-VMDvdDrive -VMName $Name -Path $ISOPath

# Set boot order to DVD first
Set-VMFirmware -VMName $Name -FirstBootDevice (Get-VMDvdDrive -VMName $Name)

# Set of the Stop Action
Set-VM -Name "Ubuntu" -AutomaticStopAction TurnOff
```

Note! Do not add the GPU at this time, you will not be able to boot to the installation.

Installing xRDP with hardware acceleration

After installing the CPU, I've noticed that the console is no longer available in the Hyper-v manager. To handle this situation we'll need to remotely connect to the system using RDP first. Obviously RDP is more Microsoft oriented but can easily be installed in Ubuntu using a predefined package repository! Introducing the The **xrdp-egfx** package. A pre-compiled number of modules that enable the remote desktop feature and best of all, includes RDP hardware acceleration! Here's how.

Remove any existing xRDP installation:

```
sudo apt purge xrdp xorgxrdp
```

Add the xRDP-eGFX repository and install the necessary packages:

```
sudo apt-add-repository ppa:saxl/xrdp-egfx
sudo apt install xrdp-egfx xorgxrdp-egfx -y
```

Enable audio redirection over RDP:

```
sudo apt install pulseaudio-module-xrdp -y
```

Before you reboot the machine, note the IP address, we'll need it to connect to the machine over RDP in the next step.

Hint! I prefer to assign a static IP address, just to give a consistant experience.

Reboot the machine, but at this stage do not login to the console. Instead on your Windows machine, use the remote desktop connection client (*mstsc.exe*) to connect to your remote Ubuntu VM. When you followed the guide to the letter you will be greeted with the xRDP, login screen.

After logging in and making sure that everything is working, shutdown the virtual machine.

Assign the GPU to the VM

Now it's time to assign the GPU to the Ubuntu virtual machine, using a simple PowerShell command should do the trick.

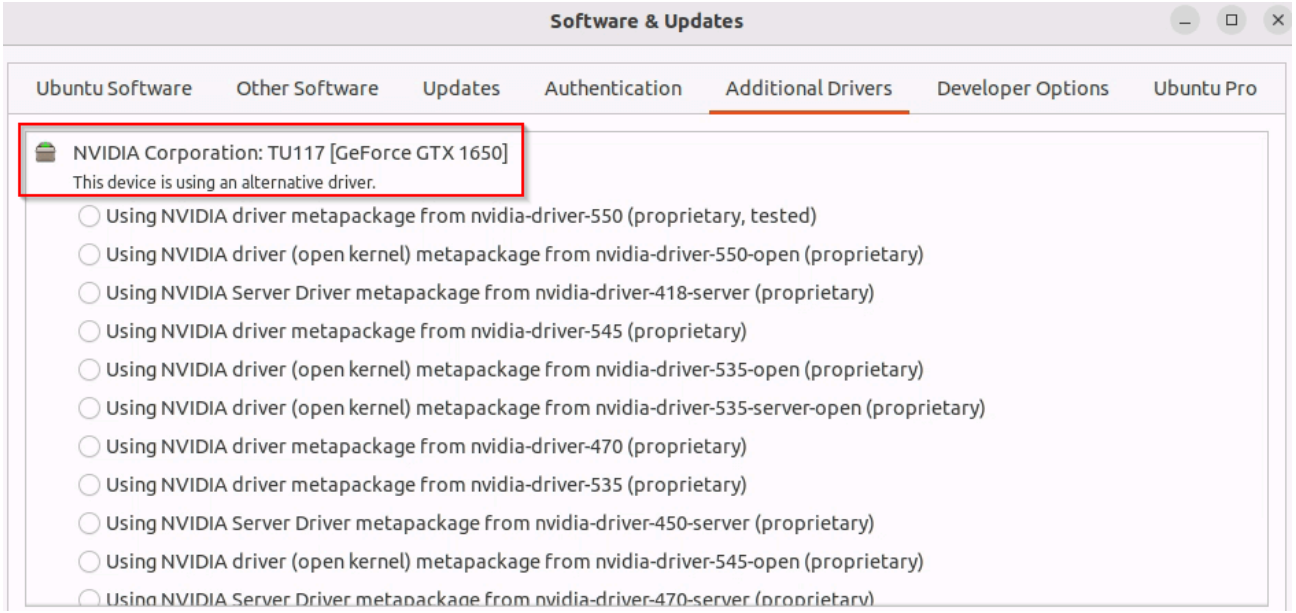
Open PowerShell as Administrator and run:

```
Add-VMAssignableDevice -LocationPath "PCIROOT(0)#PCI(0100)#PCI(0000)" -VMName
"Ubuntu"
```

On success there's no feedback. Start the virtual machine again and wait for it to boot, connect over RDP to further configure the GPU.

Validate the GPU

First we need to validate the existance of the GPU in the VM. A really easy way of doing this is to open "*Software and Updates – Additional Drivers*", the GPU, in my case the Nvidia 1650, is listed.



Or if you prefer to use the command-line:

```
superuser@lnx01:~$ sudo lshw -C display
[sudo] password for superuser:
  *-display
       description: VGA compatible controller
       product: TU117 [GeForce GTX 1650]
       vendor: NVIDIA Corporation
       physical id: 1
       bus info: pci@6746:00:00.0
       logical name: /dev/fb1
       logical name: /dev/fb0
       version: a1
       width: 64 bits
       clock: 33MHz
       capabilities: pm msi pciexpress vga_controller bus_master cap_list fb
       configuration: depth=32 driver=nouveau latency=0 mode=3840x2160 resolution=3840,2160 visual=truec
olor xres=3840 yres=2160
       resources: iomemory:f0-ef iomemory:f0-ef irq:24 memory:f9000000-f9ffffff memory:fe0000000-feffffff
ff memory:ff0000000-ff1ffffff
  *-graphics
       product: hyperv_drmdrmfb
       physical id: 1
       logical name: /dev/fb0
       capabilities: fb
       configuration: depth=32 resolution=1024,768
```

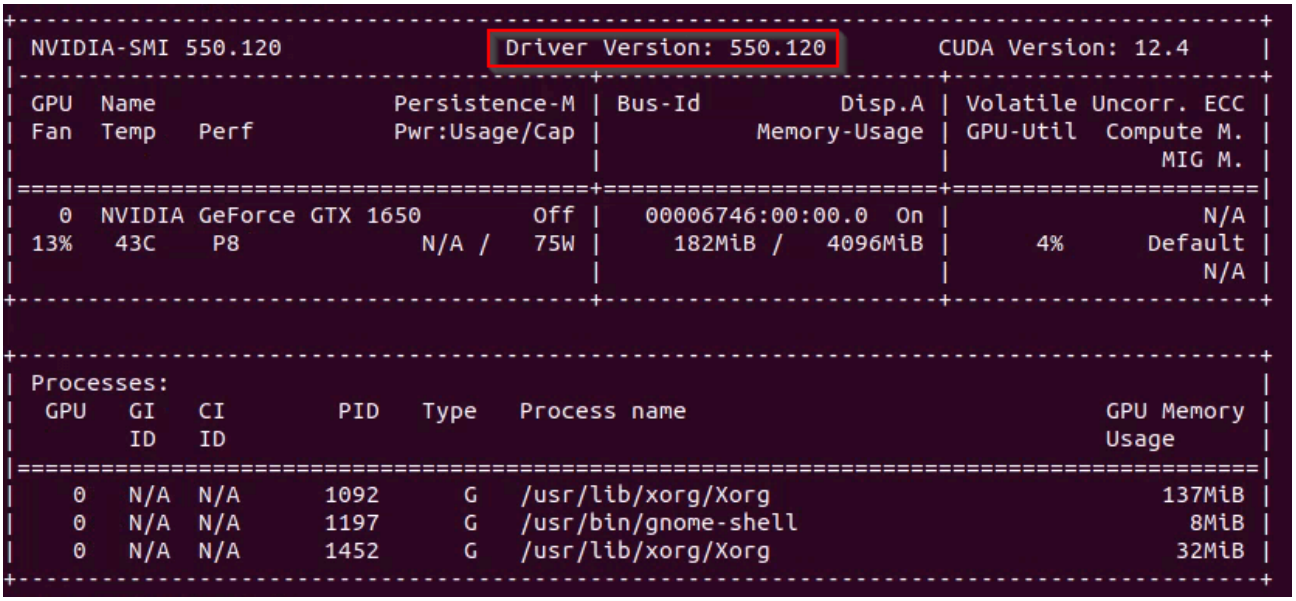
GPU Driver installation

We could use the GUI interface to select the appropriate driver, but there's a far more efficient way to install the drivers. Use the following commands in a shell to install the appropriate driver.

```
sudo apt-get remove --purge nvidia-* -y
sudo apt autoremove
sudo ubuntu-drivers autoinstall
sudo reboot
```

After the reboot and you've logged back into the machine, validate the installation of the drivers.

The output should look similar to this:



And that's it! You can now run video's or play games over RDP with full hardware acceleration and audio.

Reverting the configuration

Just in case you need to revert all the steps I've laid out in this blog, follow the exact steps below to assign the GPU back to the Operating System that is running Hyper-v.

- Shutdown the VM.
- Issue the following PowerShell commands in order on the Hyper-V host:

```
Remove-VMAssignableDevice -LocationPath "PCIROOT(0)#PCI(0100)#PCI(0000)" -VMName
Ubuntu
Mount-VMHostAssignableDevice -LocationPath "PCIROOT(0)#PCI(0100)#PCI(0000)"
Disable-PnpDevice -InstanceId
"PCI\VEN_10DE&DEV_1F82&SUBSYS_136310DE&REV_A1\4&AA66160&0&0008"
Enable-PnpDevice -InstanceId
"PCI\VEN_10DE&DEV_1F82&SUBSYS_136310DE&REV_A1\4&AA66160&0&0008"
```

Reason for first disabling and enabling the device is that I've noticed that on occasion the GPU would not be handed back to the operating system, just a safety measure.

Preventing RDP from disabling the GUI when minimized

By default, Windows pauses GUI rendering when the full screen RDP window is minimized. This can cause issues with GUI-based applications and automation tools. In this specific case it would freeze the screen and not come back. Reconnecting RDP would solve it, annoying to say the least.

To force Windows to keep rendering even when the RDP session is minimized:

- On the device you use to connect to a virtual machine, Open Registry Editor (`regedit.exe`).
- Navigate to: "*HKEY_LOCAL_MACHINE\Software\Microsoft\Terminal Server Client*".
- Create a new DWORD (32-bit) value:
  - RemoteDesktop_SuppressWhenMinimized
  - Value: 2
- Reboot the machine.

This will ensure that RDP does not pause GUI rendering, preventing application failures.

Conclusion

By setting up Discrete Device Assignment (DDA), installing the GTX 1650 drivers, enabling xrdp-egfx, and modifying RDP settings, you can create a high-performance remote desktop experience on Ubuntu inside Hyper-V, perfect for running GPU-heavy workloads.

As always, feedback is appreciated! Until next time.